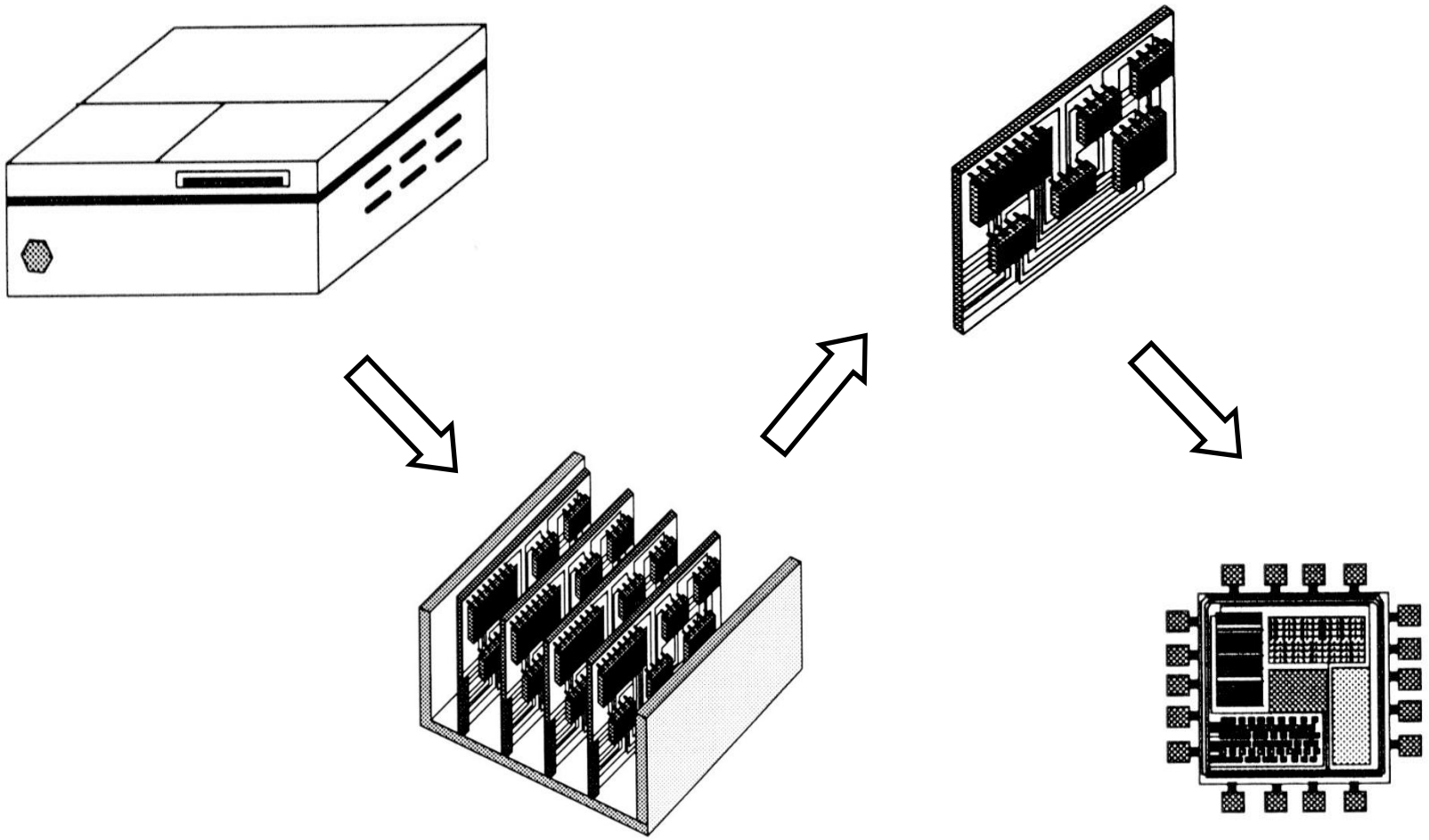
CAD for VLSI

Partitioning

System Hierarchy

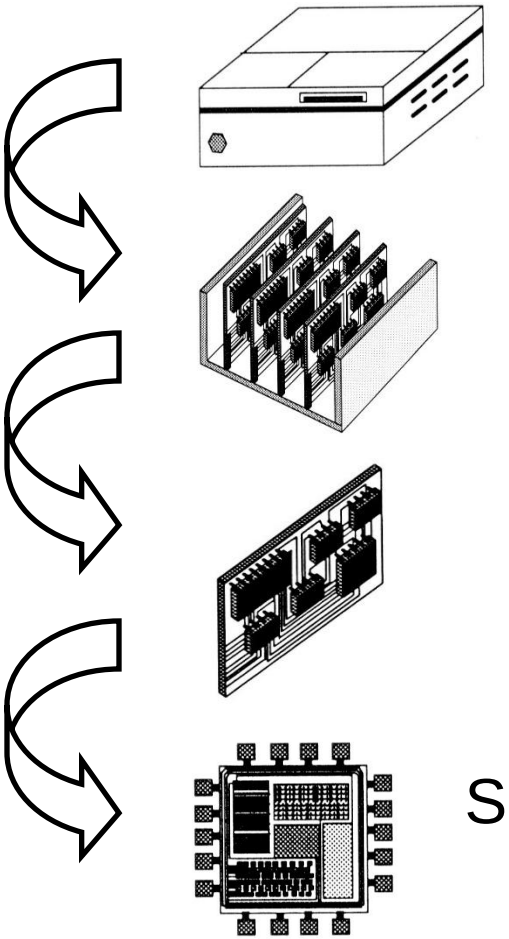


Levels of Partitioning

System Level Partitioning

Board Level Partitioning

Chip Level Partitioning



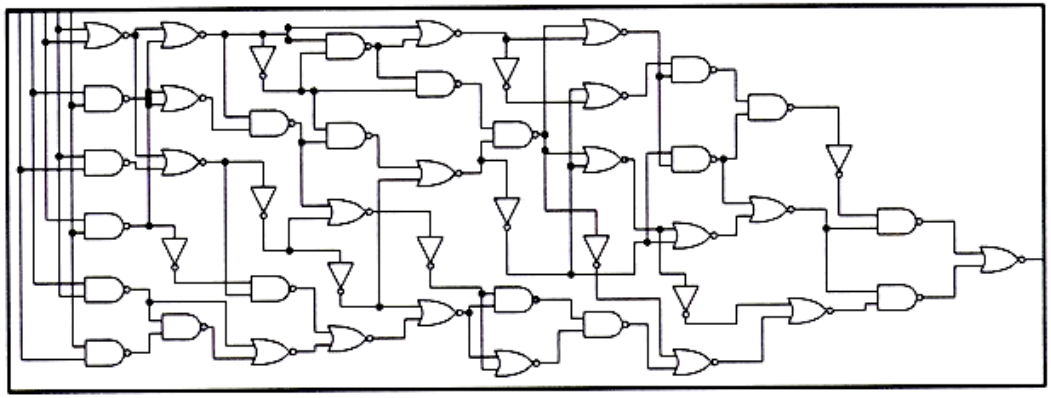
System

PCBs

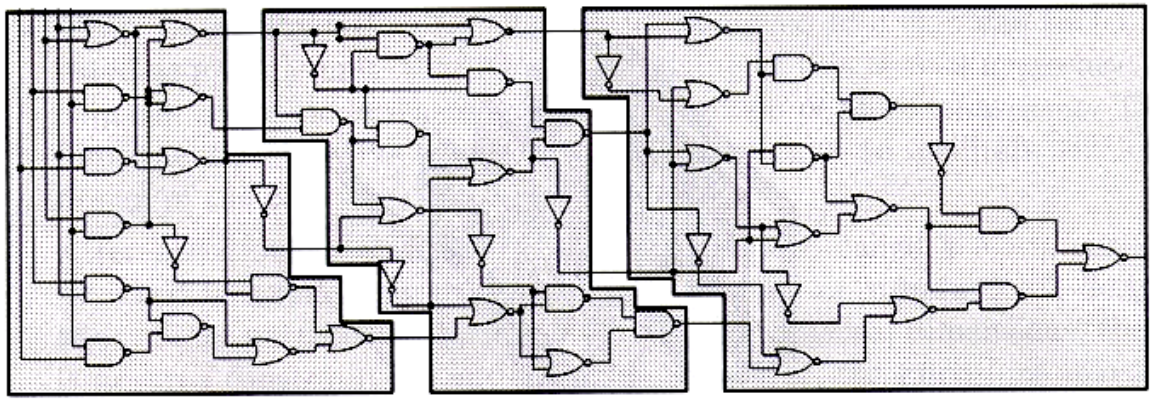
Chips

Subcircuits / Blocks

Partitioning of a Circuit



(a)



(b)

Importance of Circuit Partitioning

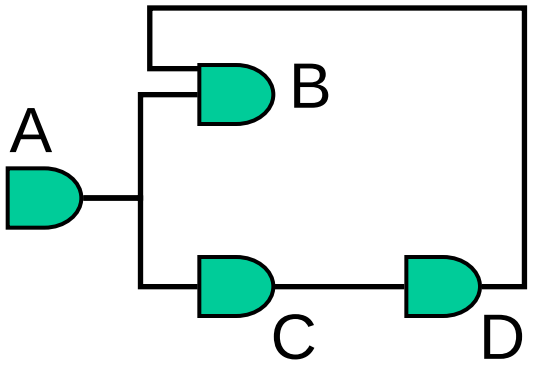
- Divide-and-conquer methodology
  - The most effective way to solve problems of high complexity
  - Ex: min-cut based placement, partitioning-based test generation,…
- System-level partitioning for multi-chip designs
  - Inter-chip interconnection delay dominates system performance.
- Circuit emulation/parallel simulation
  - Partition large circuit into multiple FPGAs (ex: Quickturn), or multiple special-purpose processors (ex: Zycad).
- Parallel CAD development
  - Task decomposition and load balancing
- In deep-submicron designs, partitioning defines local and global interconnect, and has significant impact on circuit performance
- …

Some Terminology

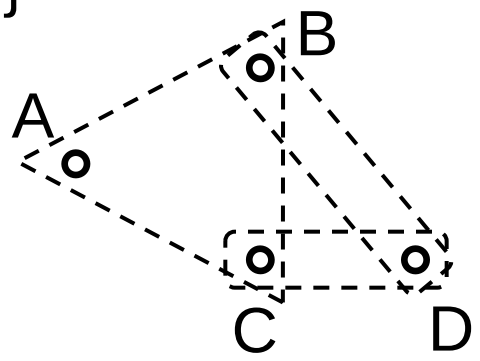
- Partitioning
  - Dividing bigger circuits into a small number of partitions (top down)
- Clustering
  - Cluster small cells into bigger clusters (bottom up)
- Covering / Technology Mapping
  - Clustering such that each partitions (clusters) have some special structure (ex: can be implemented by a cell in a cell library)
- k-way Partitioning
  - Dividing into k partitions
- Bipartitioning
  - 2-way partitioning
- Bisectioning
  - Bipartitioning such that the two partitions have the same size

Circuit Representation

- Netlist:
  - Gates: A, B, C, D
  - Nets: {A,B,C}, {B,D}, {C,D}



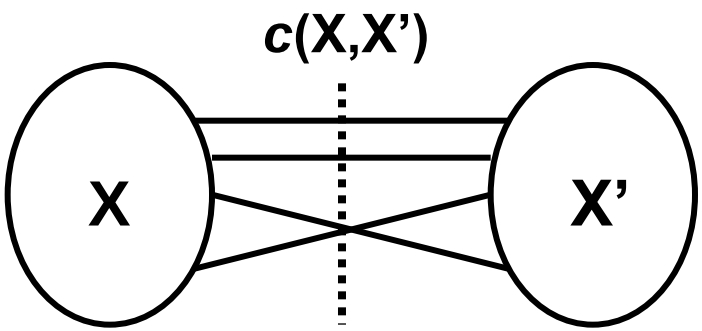
- Hypergraph:
  - Vertices: A, B, C, D
  - Hyperedges: {A,B,C}, {B,D}, {C,D}
  - Vertex label: Gate size/area
  - Hyperedge label:
    Importance of net (weight)



Circuit Partitioning Formulation

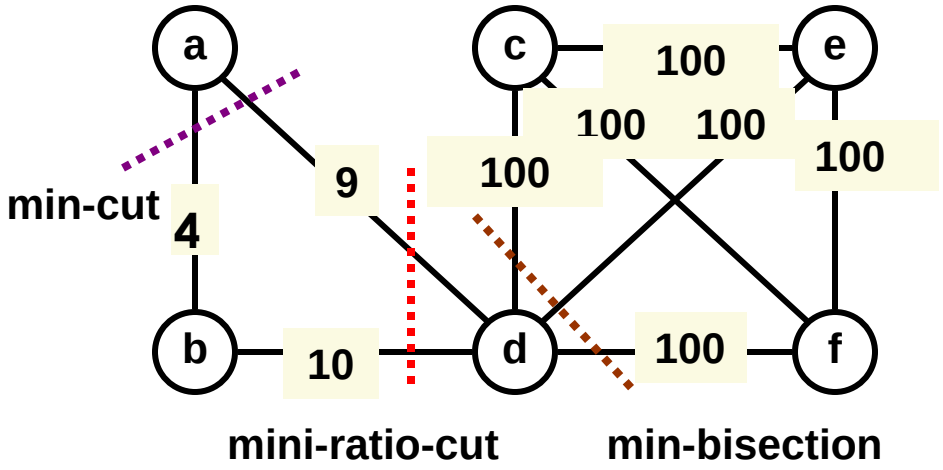
Bi-partitioning formulation:

Minimize interconnections between partitions



- Minimum cut: min $c(x, x')$
- Minimum bisection: min $c(x, x')$ with $|x| = |x'|$
- Minimum ratio-cut: min $c(x, x') / |x||x'|$

A Bi-partitioning Example



Min-cut: cut size = 13

Min-bisection cut: cut size = 300

Min-ratio-cut: cut size = 19

Ratio-cut helps to identify natural clusters

Circuit Partitioning Formulation

- General multi-way partitioning formulation:
  - Partition a network N into N_1, N_2, …, N_k such that
    - Each partition has an area constraint

$$\sum_{v \in N_i} a(v) \leq A_i$$

    - Each partition has an I/O constraint

$$c(N_i, N - N_i) \leq I_i$$

  - Minimize the total interconnection:

$$\sum_{N_i} c(N_i, N - N_i)$$

Partitioning Algorithms

- Iterative partitioning algorithms
- Spectral based partitioning algorithms
- Deterministic vs. probabilistic algorithms
- Net partitioning vs. module partitioning
- Multi-way partitioning
- Multi-level partitioning
- Further study in partitioning techniques (timing-driven …)

Iterative Partitioning Algorithms

- Greedy iterative improvement method
  - [Kernighan-Lin 1970]
  - [Fiduccia-Mattheyses 1982]
  - [Krishnamurthy 1984]
- Simulated Annealing
  - [Kirkpartrick-Gelatt-Vecchi 1983]
  - [Greene-Supowit 1984]

Kernighan-Lin Algorithm

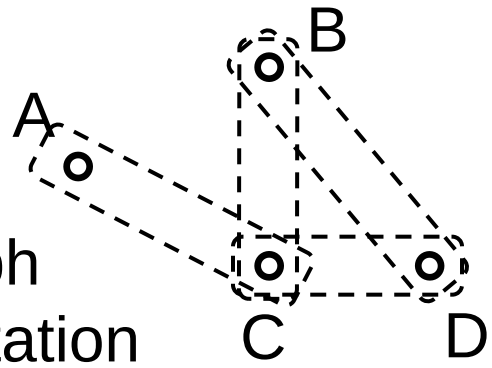
"An Efficient Heuristic Procedure for Partitioning Graphs"

The Bell System Technical Journal

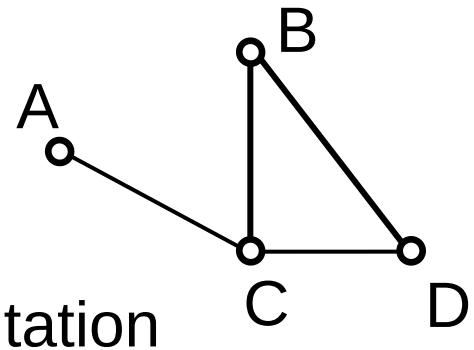
49(2):291-307, 1970

Restricted Partition Problem

- Restrictions:
  - For Bisectioning of circuit
  - Assume all gates are of the same size
  - Works only for 2-terminal nets
- If all nets are 2-terminal,
  the hypergraph is called a graph



Hypergraph Representation



Graph Representation

Problem Formulation

- Input: A graph with
  - Set vertices V ($|V| = 2n$)
  - Set of edges E ($|E| = m$)
  - Cost c_{AB} for each edge $\{A, B\}$ in E
- Output: 2 partitions X & Y such that
  - Total cost of edges cut is minimized
  - Each partition has n vertices

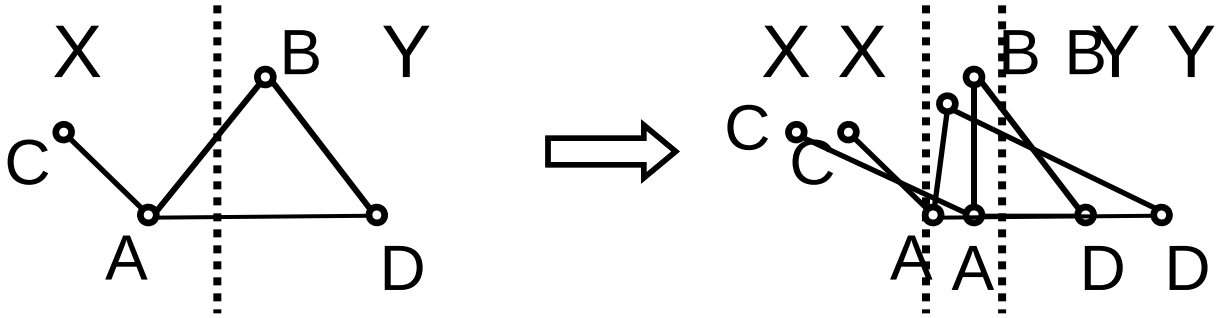
- This problem is NP-Completeness!!!!!

A Trivial Approach

- Try all possible bisections, find the best one
- If there are *2n* vertices,
  # of possibilities = $C_n^{2n} / 2$ = $(2n)! / (2 \times (n!^2)) = n^{O(n)}$
- For 4 vertices (A,B,C,D), 3 possibilities
  1. X={A,B} & Y={C,D}
  2. X={A,C} & Y={B,D}
  3. X={A,D} & Y={B,C}
- For 100 vertices, 5×10^{28} possibilities
  - Need 1.59×10^{12} years if someone can try 1G (10^9) possibilities per second

Idea of KL Algorithm

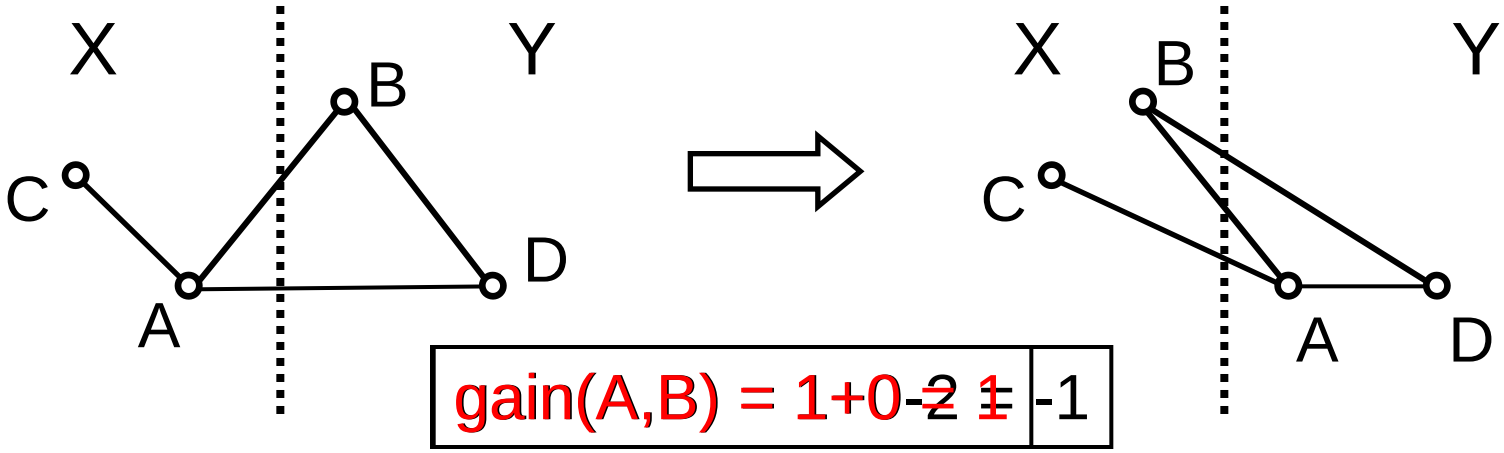
- D_A = Decrease in cut value if moving A
  - External cost (connection) E_A – Internal cost I_A
  - Moving node a from block A to block B would increase the value of the cutset by E_A and decrease it by I_A



$D_A = 2-1 = 1$
$D_B = 1-1 = 0$

Idea of KL Algorithm

- Note that we want to balance two partitions
- If switch A & B, gain(A,B) = $D_A+D_B-2c_{AB}$
  - c_{AB} : edge cost for AB

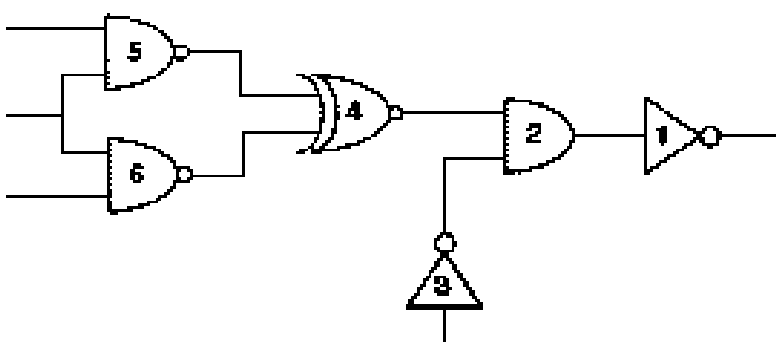


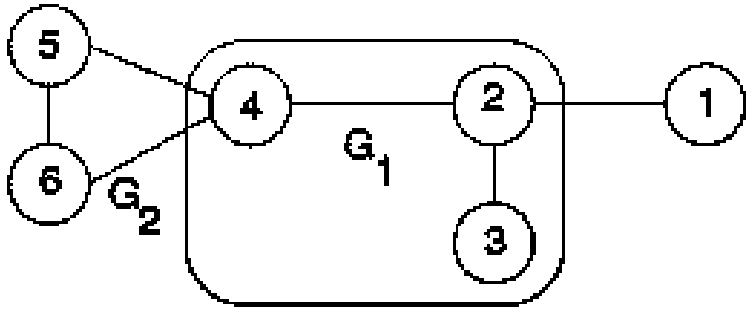
gain(A,B) = 1+0-2 1 = -1

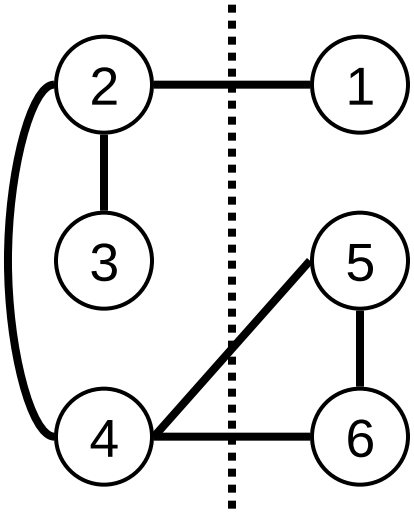
Idea of KL Algorithm

- Start with any initial legal partitions X and Y
- A pass (exchanging each vertex exactly once) is described below:
  1. For i := 1 to n do
     From the unlocked (unexchanged) vertices,
     choose a pair (A,B) s.t. gain(A,B) is largest
     Exchange and Lock A, B
     Let g_i = gain(A,B)
  2. Find the k s.t. $G=g_1+...+g_k$ is maximized
  3. Switch the first k pairs
- Repeat the pass until there is no improvement (G=0)

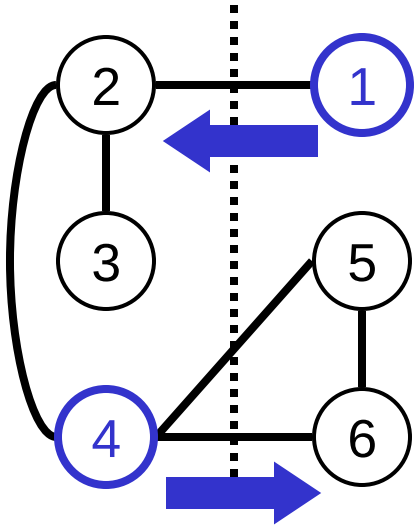
KL Example – 1





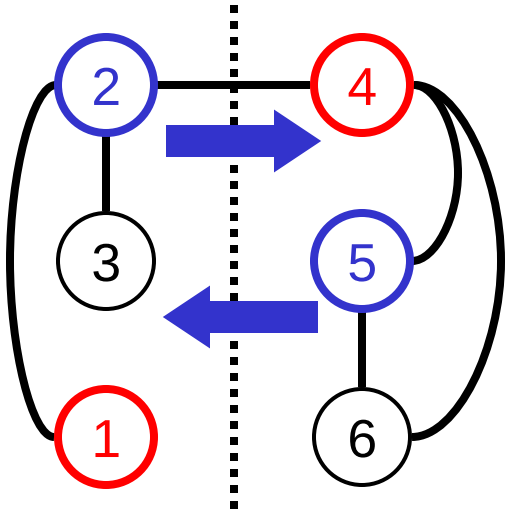


KL Example – 2



- $D_1 = 1-0 = 1$
- $D_2 = 1-2 = -1$
- $D_3 = 0-1 = -1$
- $D_4 = 2-1 = 1$
- $D_5 = 1-1 = 0$
- $D_6 = 1-1 = 0$
- gain(2, 1) = -1 + 1 – 2 = -2
- gain(2, 5) = -1 + 0 – 0 = -1
- gain(2, 6) = -1 + 0 – 0 = -1
- gain(3, 1) = -1 + 1 – 0 = 0
- gain(3, 5) = -1 + 0 – 0 = -1
- gain(3, 6) = -1 + 0 – 0 = -1
- gain(4, 1) = 1 + 1 – 0 = 2 g_1
- gain(4, 5) = 1 + 0 – 2 = -1
- gain(4, 6) = 1 + 0 – 2 = -1

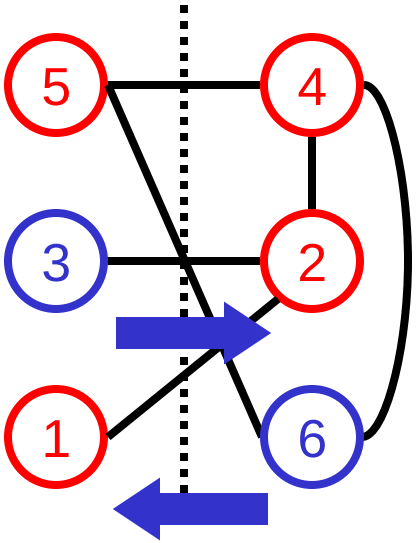
KL Example – 3

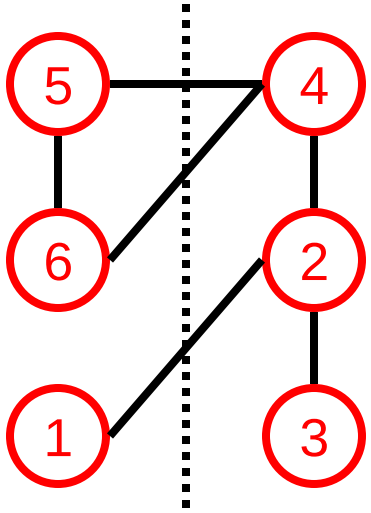


- $D_2 = 1 - 2 = -1$
- $D_3 = 0 - 1 = -1$
- $D_5 = 0 - 2 = -2$
- $D_6 = 0 - 2 = -2$
- gain(2, 5) = -1 + -2 – 0 = -3 g_2
- gain(2, 6) = -1 + -2 – 0 = -3
- gain(3, 5) = -1 + -2 – 0 = -3
- gain(3, 6) = -1 + -2 – 0 = -3

KL Example – 4

- $D_3 = 1 - 0 = 1$
- $D_6 = 1 - 1 = 0$
- gain(3, 6) = 1 – 0 – 0 = 1 g_3





$g_1 = 2,\ g_2 = -3,\ g_3 = 1$

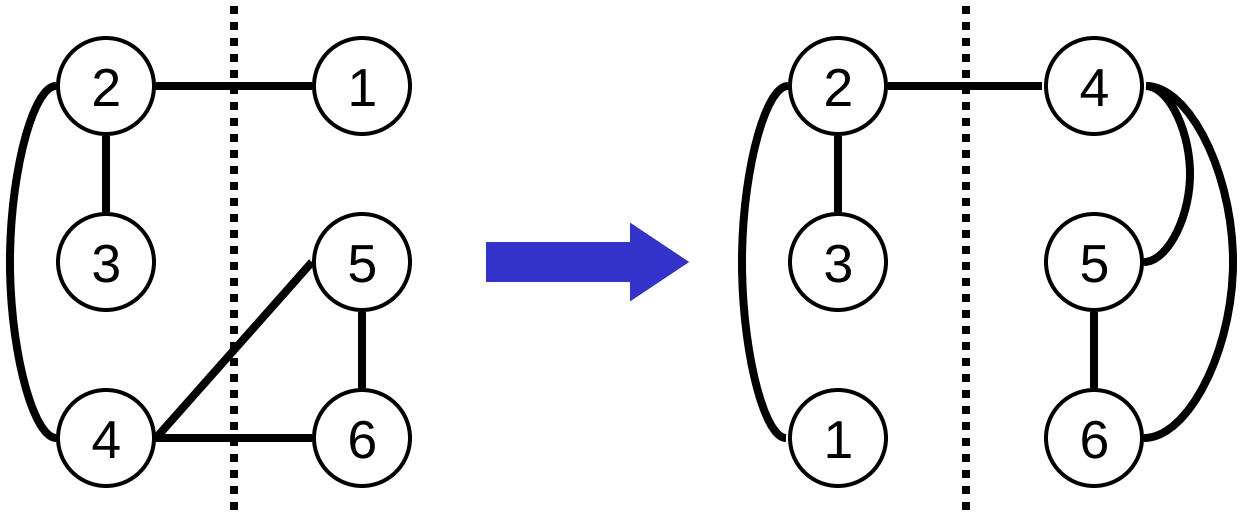
Then ???

We need to find k

- k = 1: $g_1 = 2$
- k = 2: $g_1 + g_2 = -1$
- k = 3: $g_1 + g_2 + g_3 = 0$

KL Example – 5

- Since k = 1, we only take 1 move → swap (4, 1)



- Finally, we get the result?

NO! We only pass 1 iteration!

- How should I do to get an end?

Time Complexity of KL

- For each pass KL
  - $O(n^2)$ time to find the best pair to exchange
  - n pairs exchanged
  - Total time is $O(n^3)$ per pass
- Better implementation can get $O(n^2 \log n)$ time per pass
- Number of passes is usually small

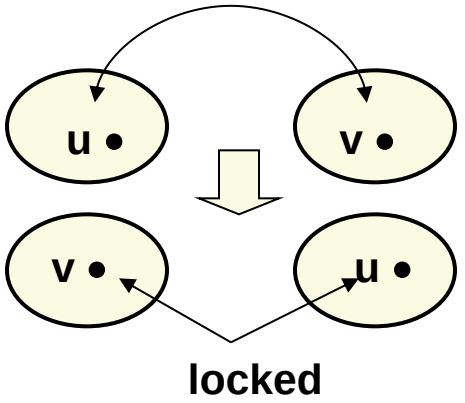
Recap of KL Algorithm

Pair-wise exchange of nodes to reduce cut size

Allow cut size to increase temporarily within a pass

Compute the gain of a swap

- Repeat
  - Perform a feasible swap of max gain
  - Mark swapped nodes “locked”
  - Update swap gains
- Until no feasible swap



Find max prefix partial sum in gain sequence $g_1, g_2, \ldots, g_m$

Make corresponding swaps permanent

Start another pass if current pass reduces the cut size (usually converge after a few passes)

A Useful Survey Paper

- Charles Alpert and Andrew Kahng, “Recent Directions in Netlist Partitioning: A Survey”, Integration: the VLSI Journal, 19(1-2), 1995, pp. 1-81

Fiduccia-Mattheyses Algorithm

"A Linear-time Heuristics
for Improving Network Partitions"
19th DAC, pages 175-181, 1982.

Features of FM Algorithm

- Modification of KL Algorithm:
  - Can handle non-uniform vertex weights (areas)
  - Allow unbalanced partitions
  - Extended to handle hypergraphs
  - Clever way to select vertices to move, run much faster

Problem Formulation

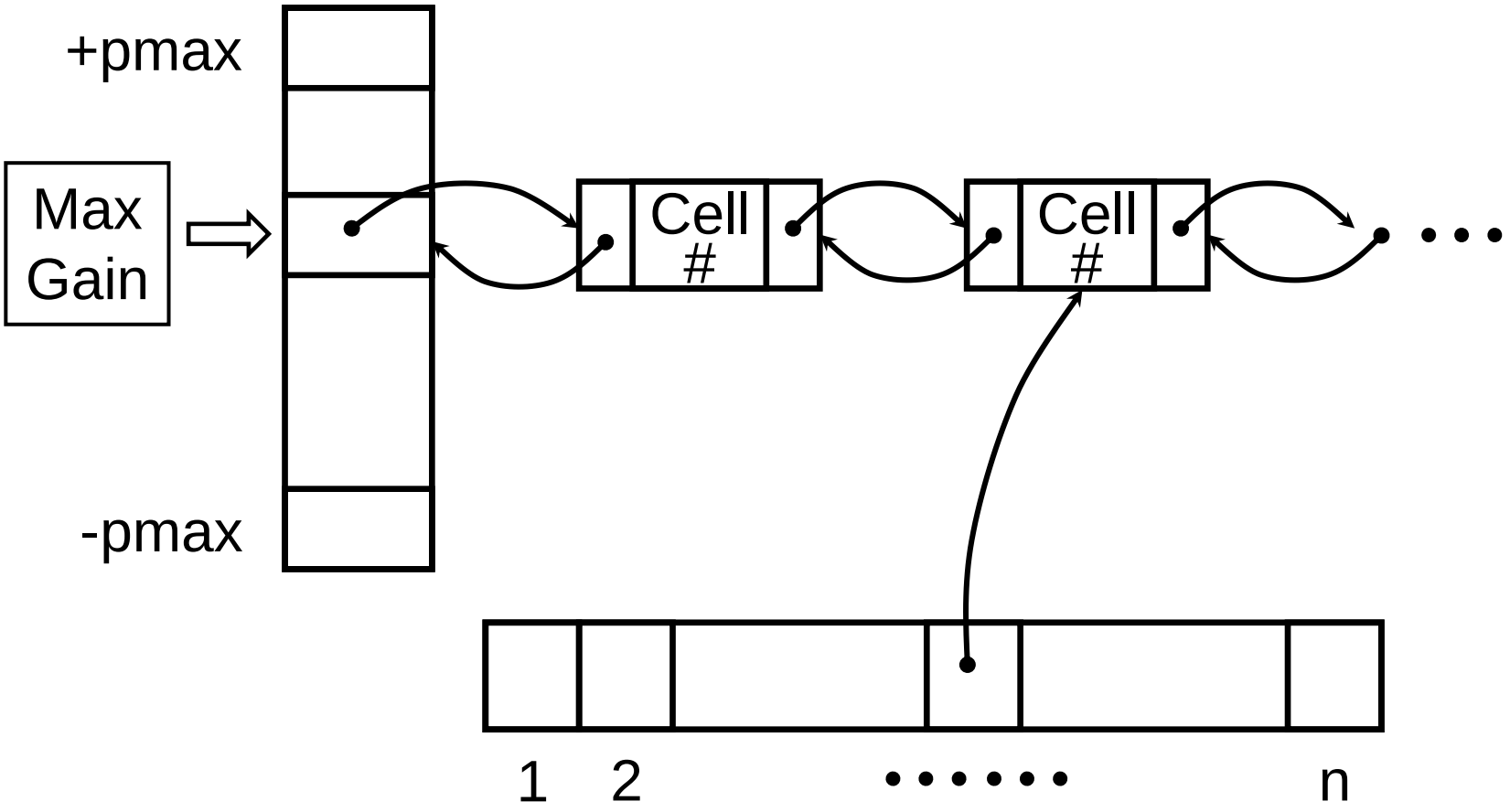
- Input: A hypergraph with
  - Set vertices V. (|V| = n)
  - Set of hyperedges E (total # pins in netlist = p)
  - Area a_u for each vertex u in V
  - Cost c_e for each hyperedge in e
  - An area ratio r
- Output: 2 partitions X & Y such that
  - Total cost of hyperedges cut is minimized
  - area(X) / (area(X) + area(Y)) is about r

- This problem is NP-Complete!!!!!

Ideas of FM Algorithm

- Similar to KL:
  - Work in passes
  - Lock vertices after moved
  - Actually, only move those vertices up to the maximum partial sum of gain
- Difference from KL:
  - Not exchanging pairs of vertices
    - Move only one vertex at each time
  - The use of gain bucket data structure

Gain Bucket Data Structure



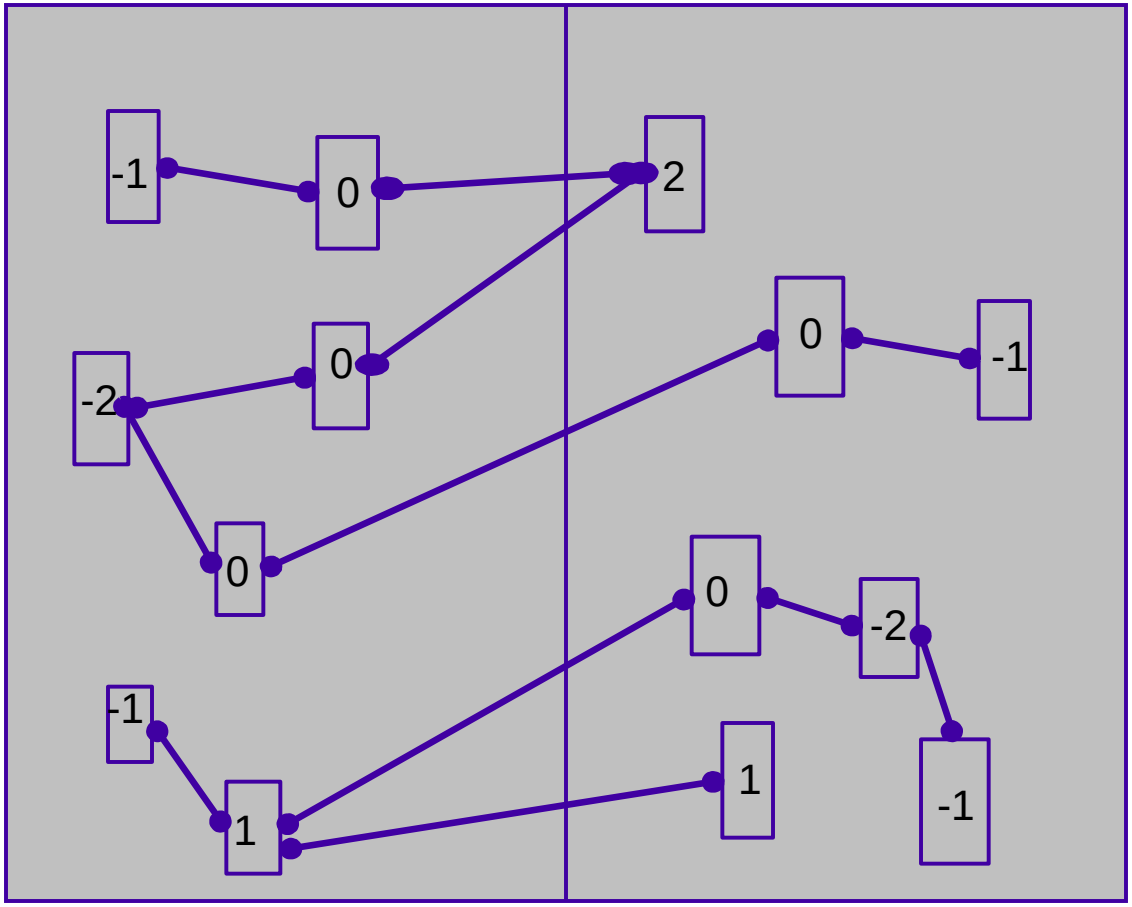
FM Partitioning

- Moves are made based on object gain
- Object Gain: The amount of change in cut crossings that will occur if an object is moved from its current partition into the other partition
- A pass description

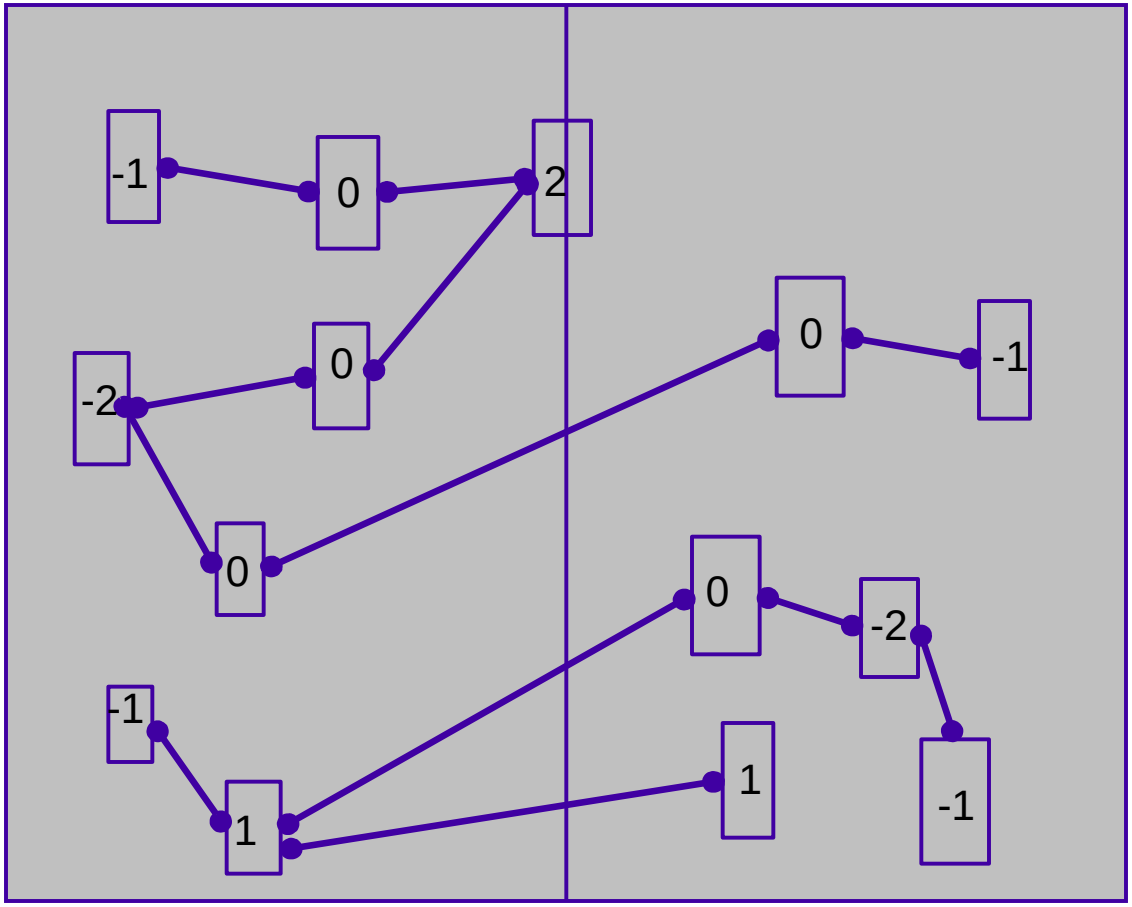
  While there is unlocked object

  1. Each object is assigned a gain
  2. Objects are put into a sorted gain list
  3. The object with the highest gain from the larger of the two sides is selected and moved
  4. The moved object is "locked"
  5. Gains of "touched" objects are recomputed
  6. Gain lists are resorted
- Repeat the pass until there is no improvement

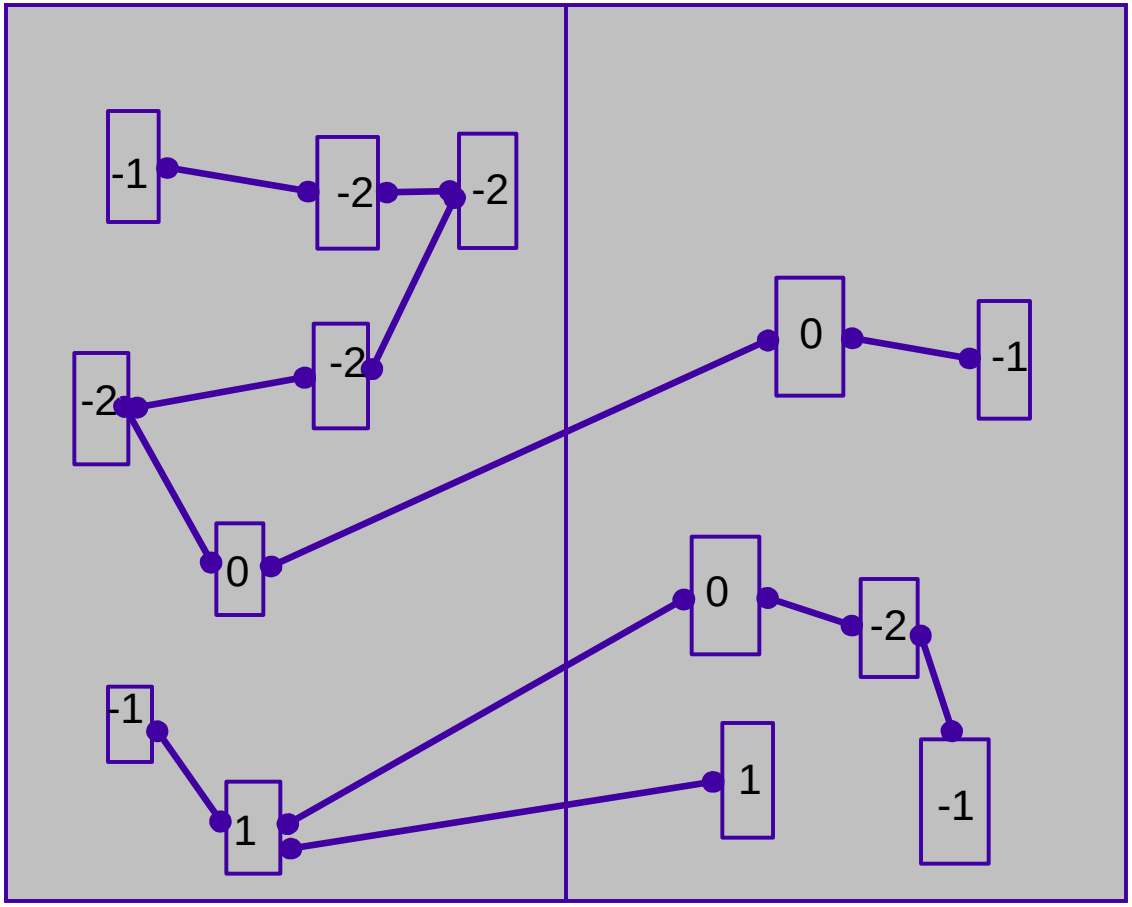
FM Partitioning



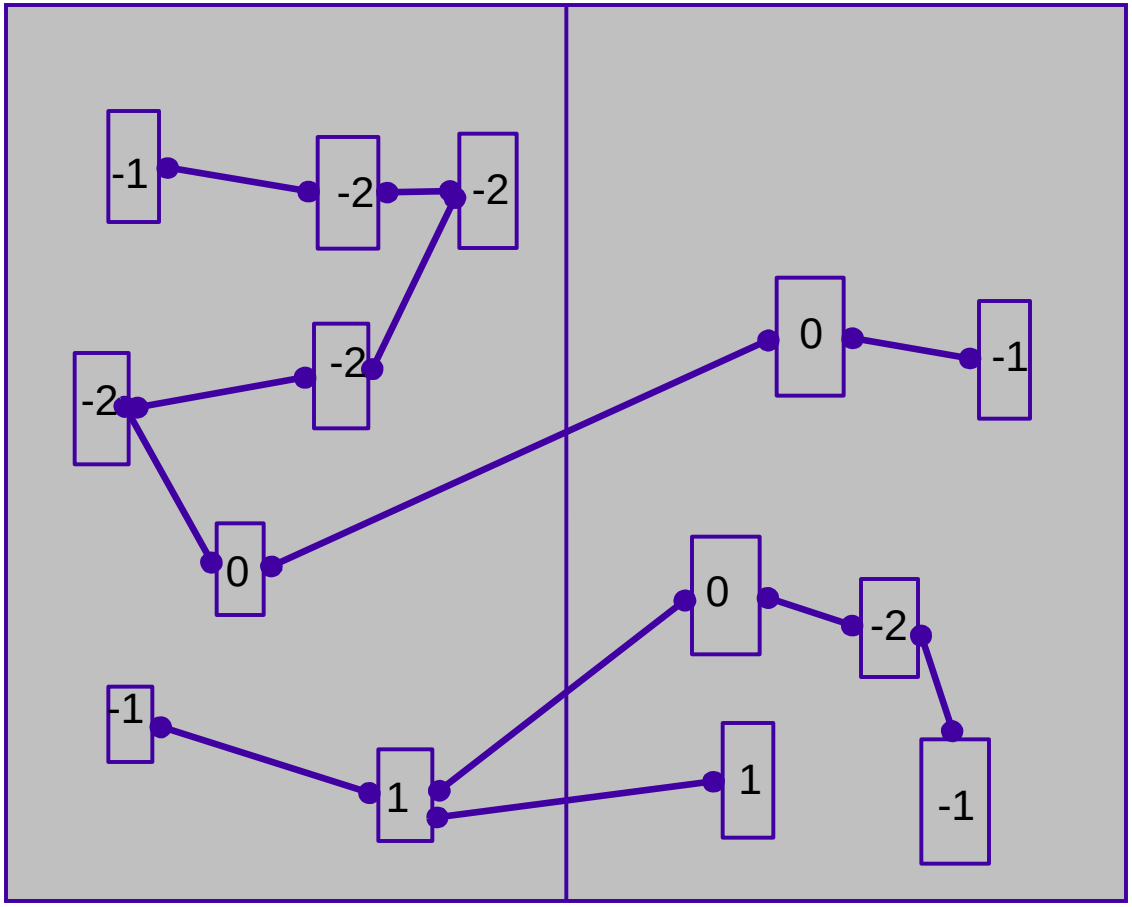
FM Partitioning



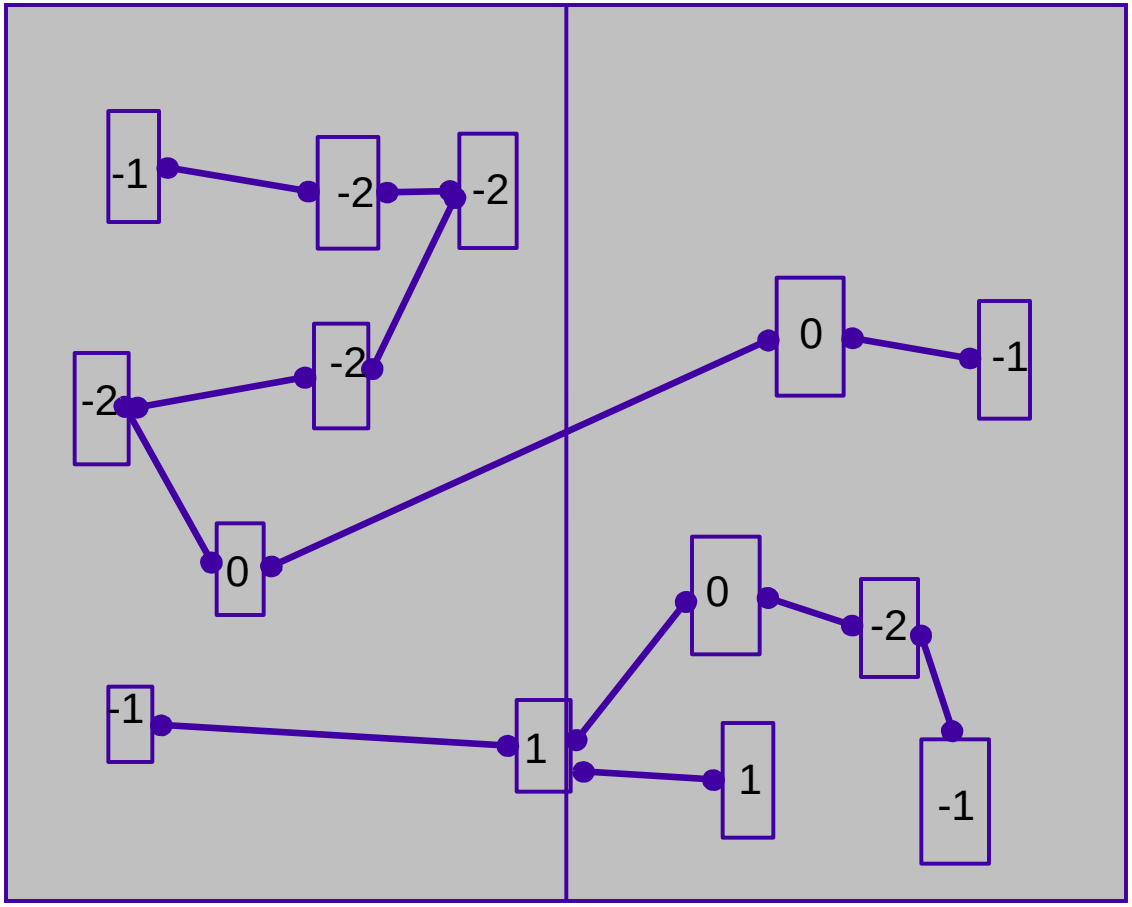
FM Partitioning



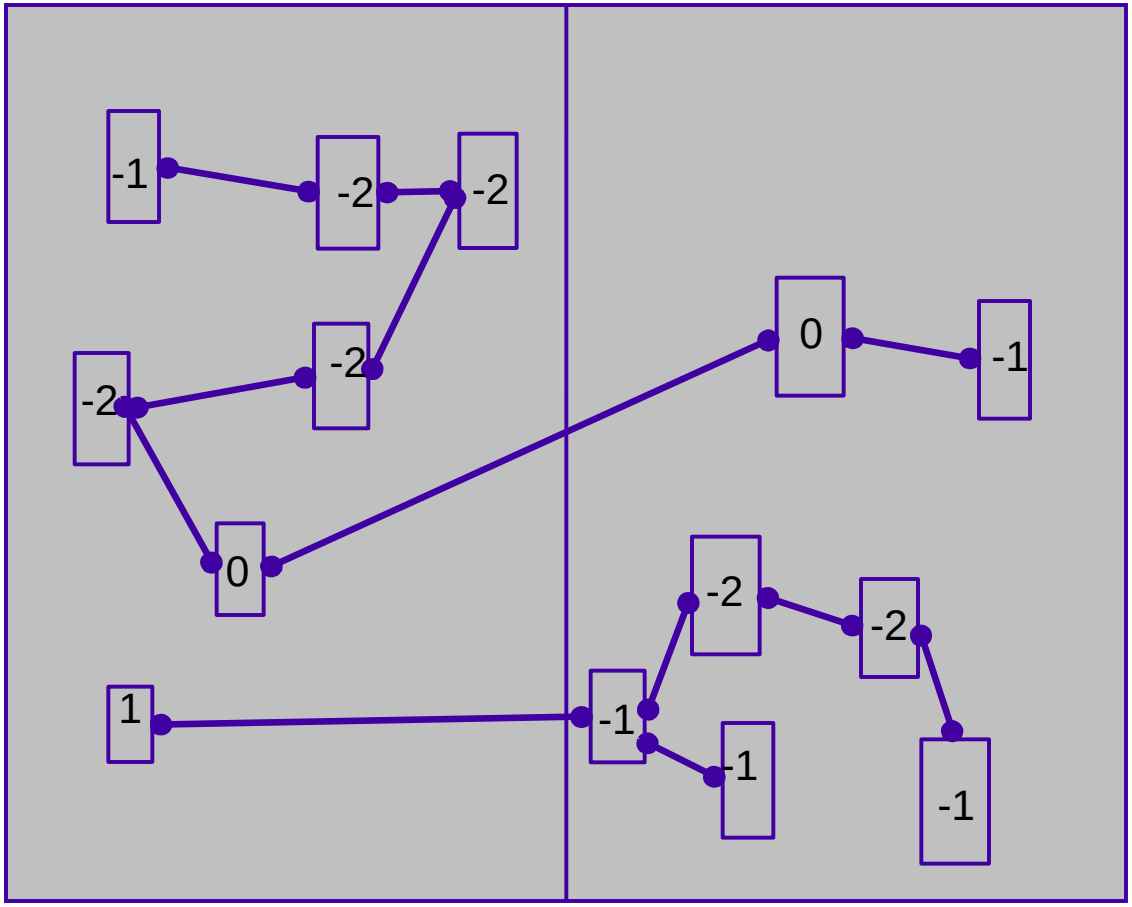
FM Partitioning



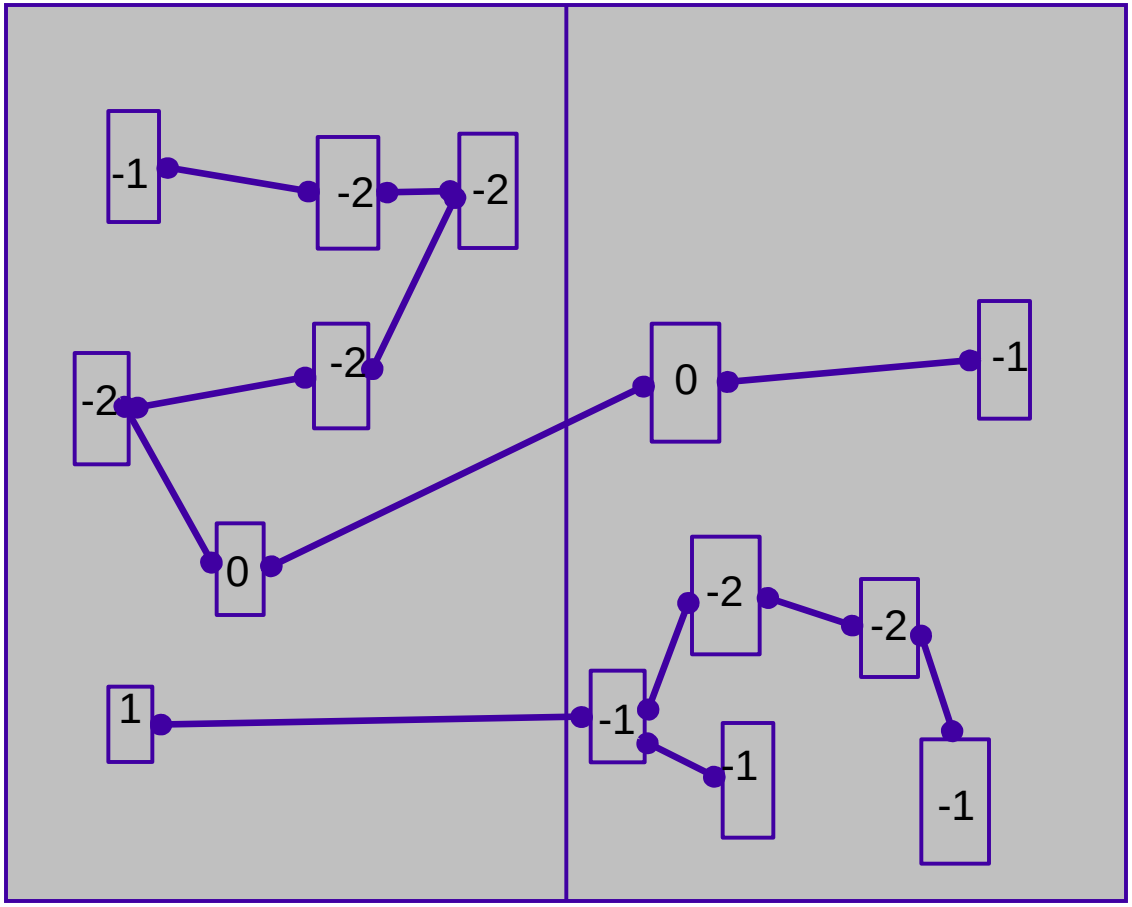
FM Partitioning



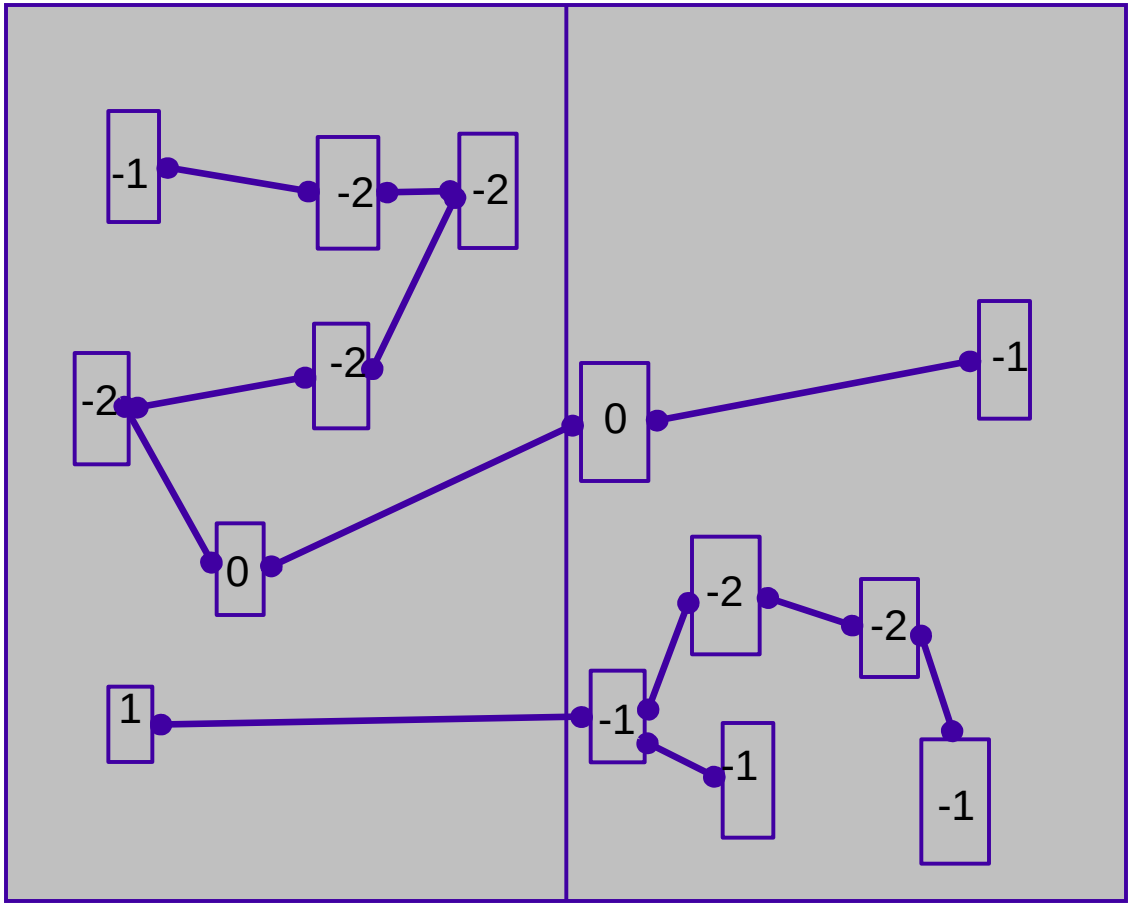
FM Partitioning



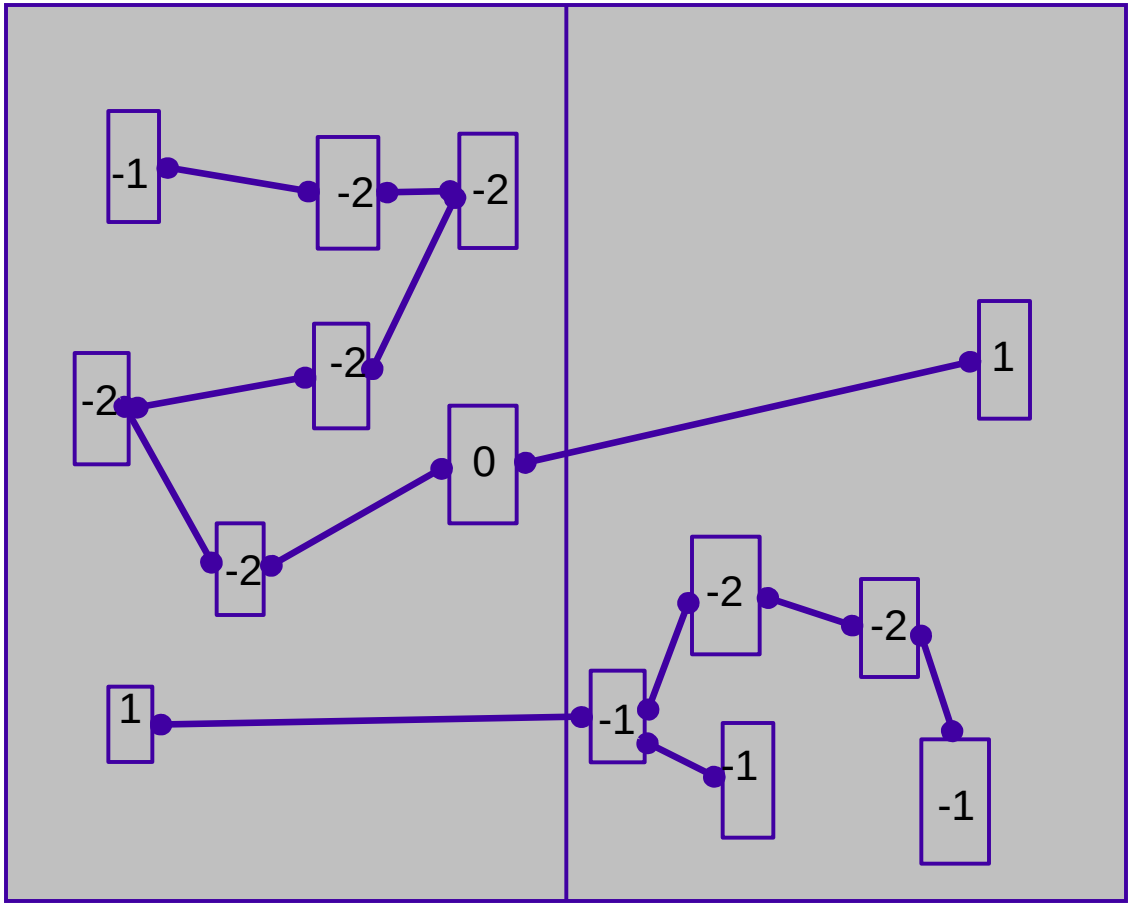
FM Partitioning



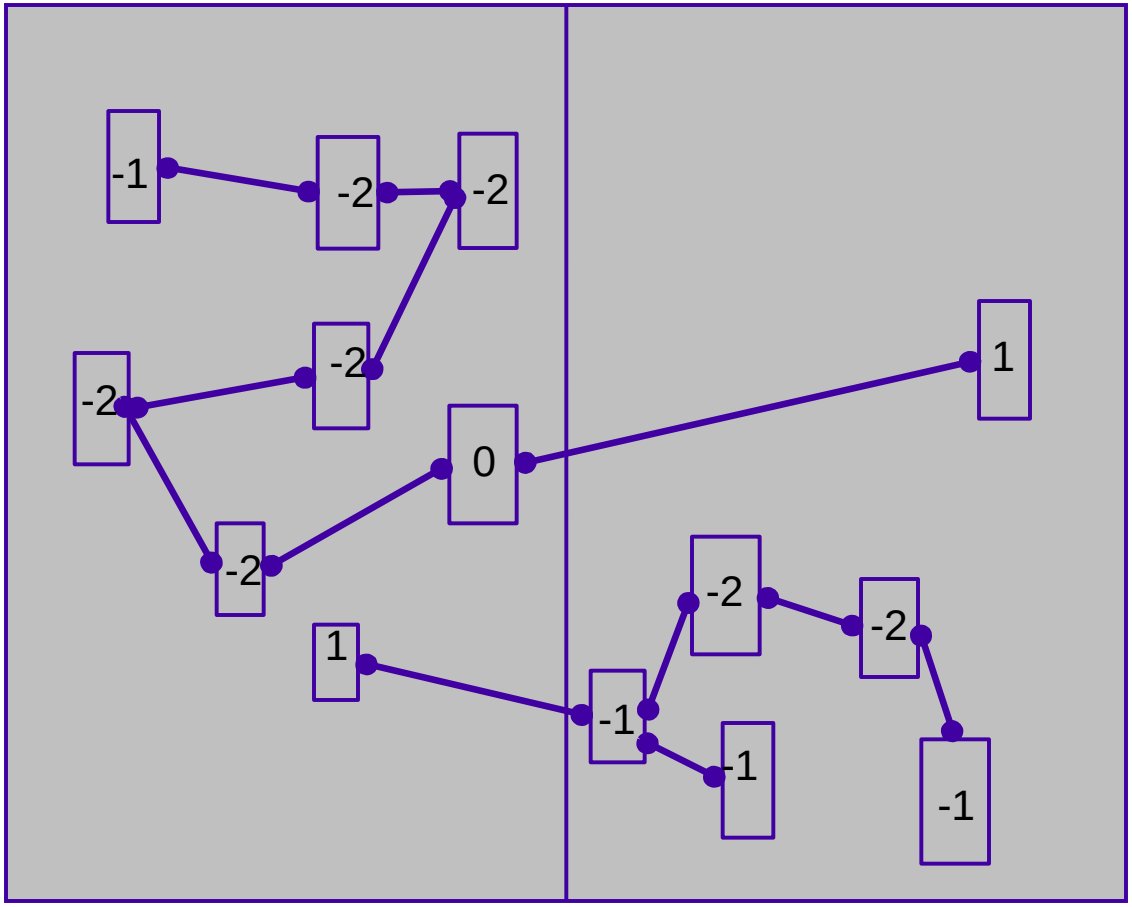
FM Partitioning



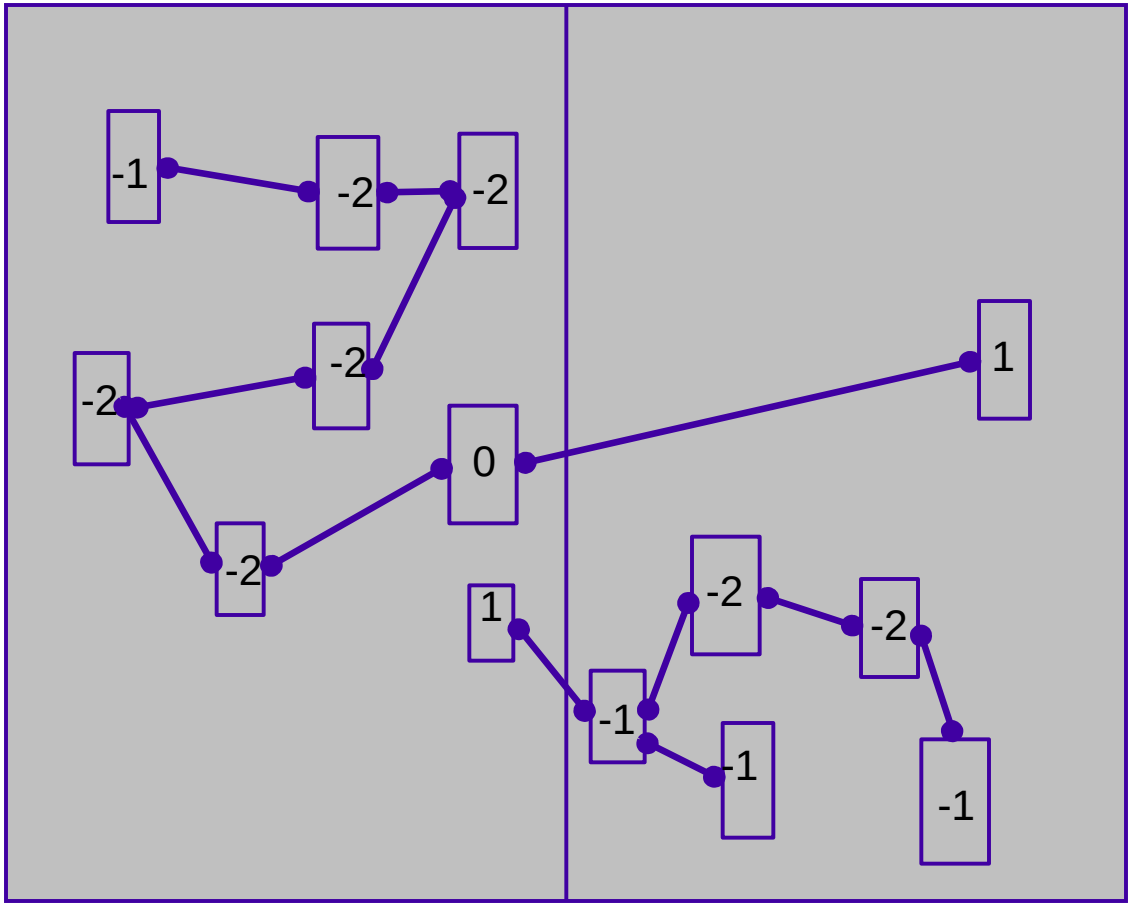
FM Partitioning



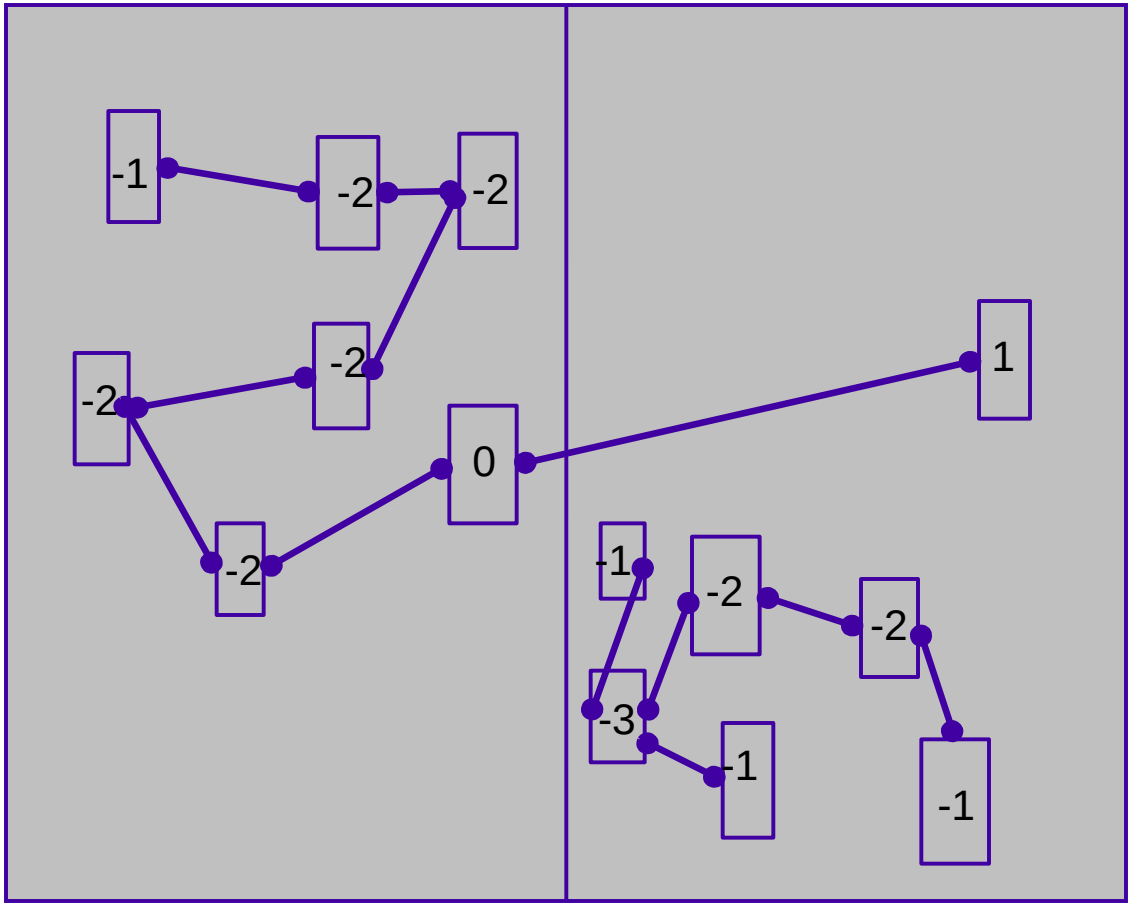
FM Partitioning



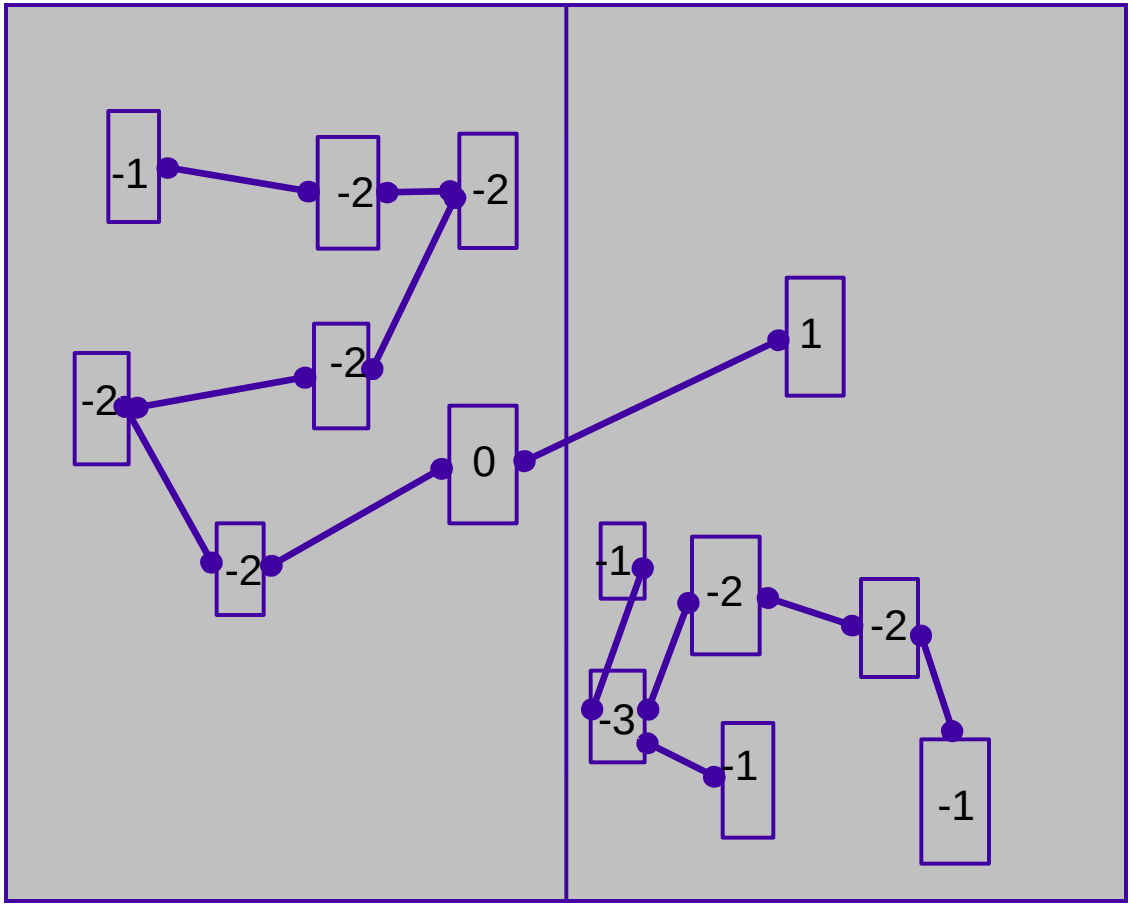
FM Partitioning



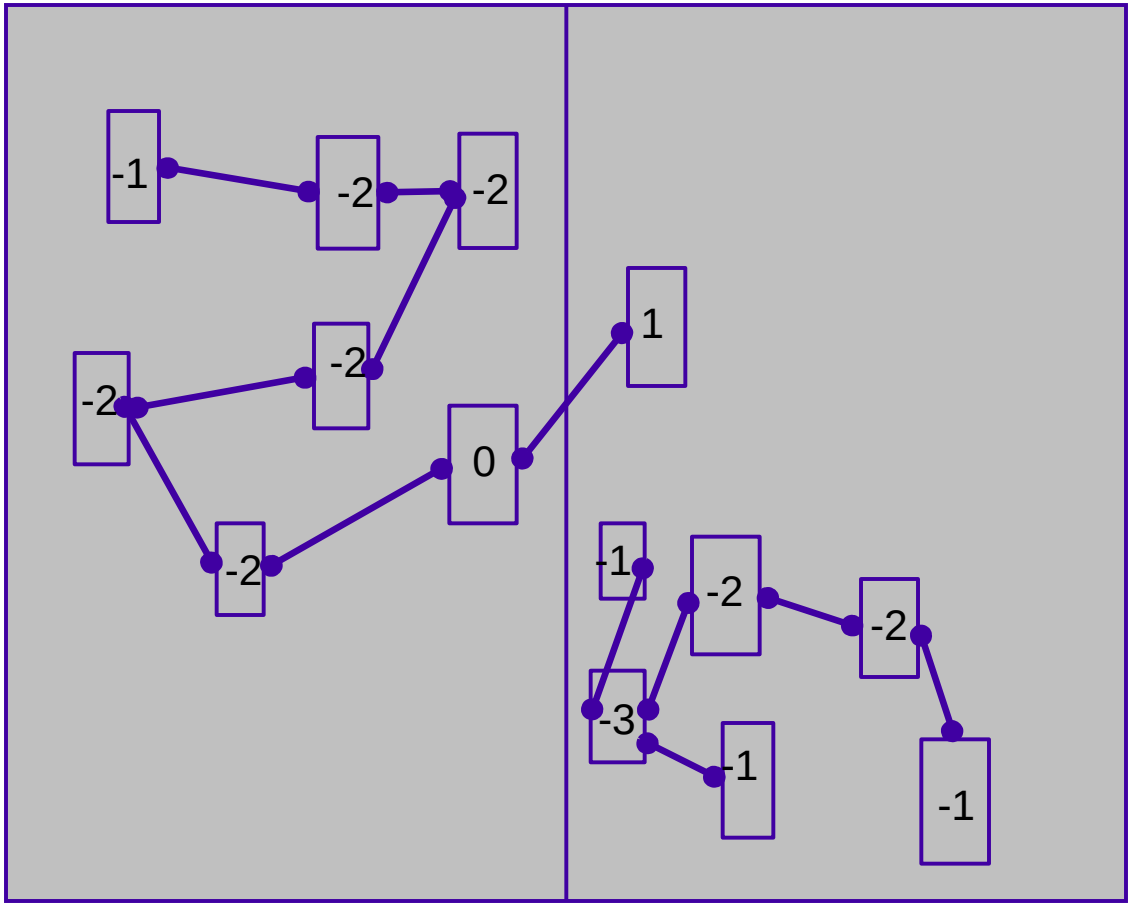
FM Partitioning



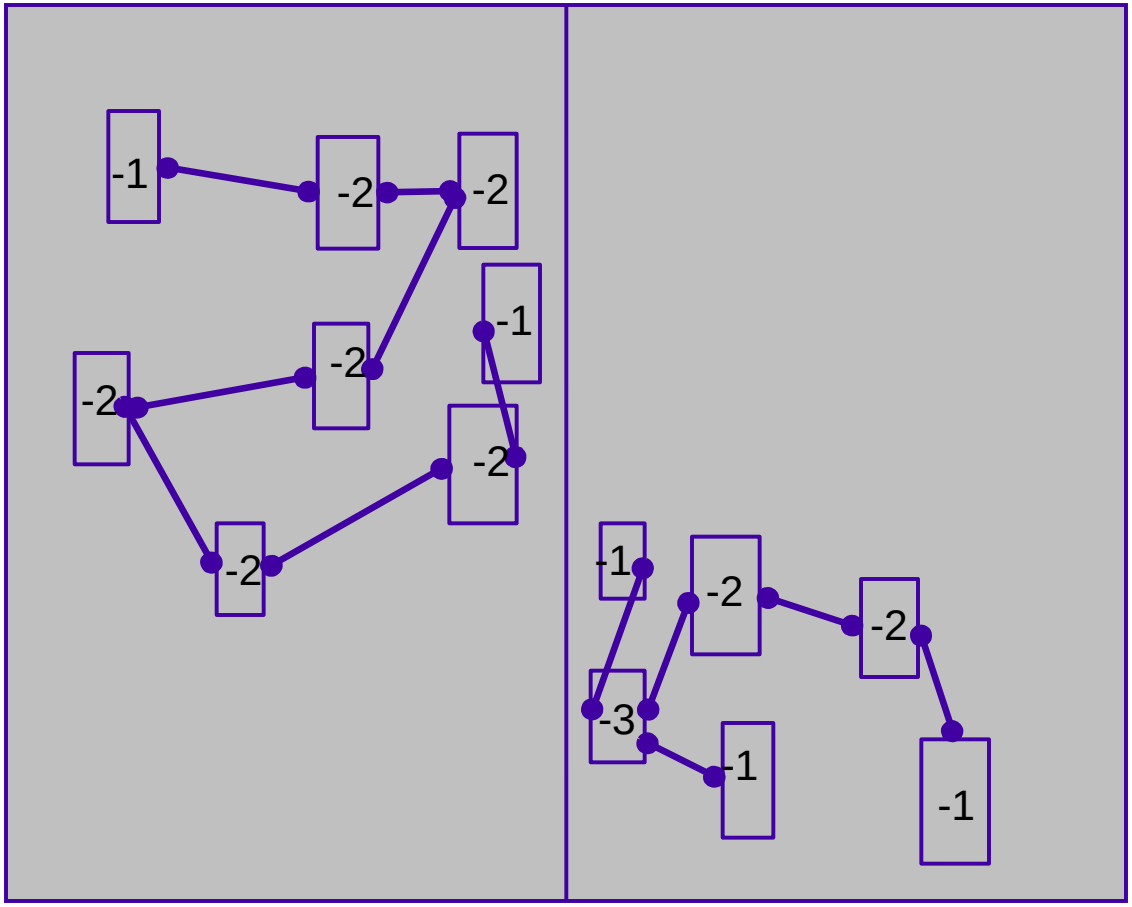
FM Partitioning



FM Partitioning



FM Partitioning



Time Complexity of FM

- For each pass,
  - Constant time to find the best vertex to move
  - After each move, time to update gain buckets is proportional to degree of vertex moved
  - Total time is O(p), where p is total number of pins
- Number of passes is usually small

Extension by Krishnamurthy

"An Improved Min-Cut Algorithm for Partitioning VLSI Networks",

IEEE Trans. Computer,

33(5):438-446, 1984.

Tie-Breaking Strategy

- For each vertex, instead of having a gain bucket, a gain vector is used
- Gain vector is a sequence of potential gain values corresponding to numbers of possible moves into the future
- Therefore, r^{th} entry looks r moves ahead
- Time complexity is O(pr), where r is max # of look-ahead moves stored in gain vector

- If ties still occur, some researchers observe that LIFO order improves solution quality

Ratio Cut Objective by Wei and Cheng

"Towards Efficient Hierarchical Designs by Ratio Cut Partitioning",

ICCAD, pages 1:298-301, 1989.

Ratio Cut Objective

- It is not desirable to have some pre-defined ratio on the partition sizes
- Wei and Cheng proposed the Ratio Cut objective
- Try to locate natural clusters in circuit and force the partitions to be of similar sizes at the same time
- Ratio Cut $R_{XY} = C_{XY}/(|X| \bullet |Y|)$
- A heuristic based on FM was proposed

Sanchis Algorithm

"Multiple-way Network Partitioning",
IEEE Trans. Computers,
38(1):62-81, 1989.

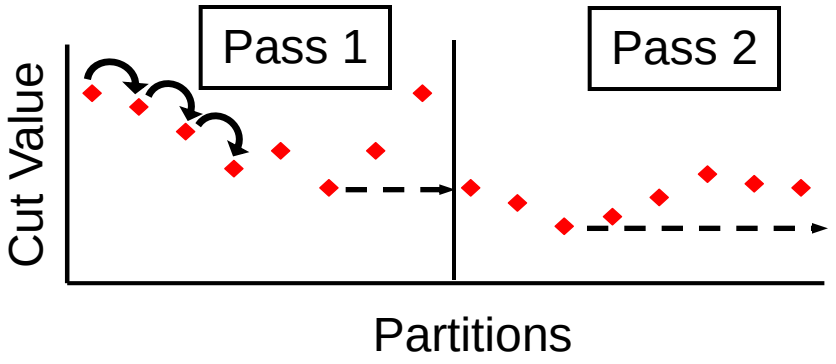
Multi-Way Partitioning

- Dividing into more than 2 partitions
- Algorithm by extending the idea of FM + Krishnamurthy

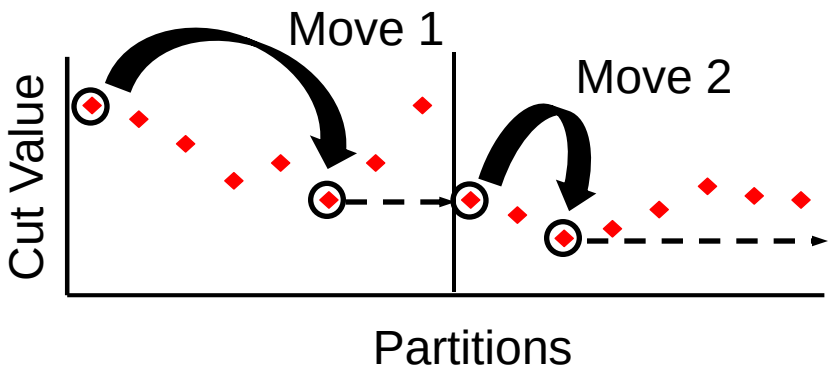
Partitioning: Simulated Annealing

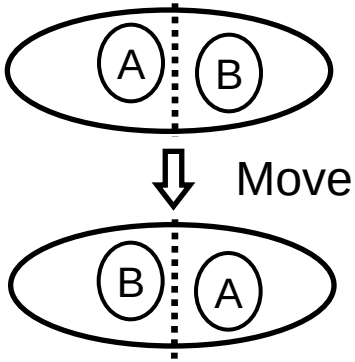
Greedy Nature of KL & FM

- KL and FM are *almost* greedy algorithms



- Purely greedy if we consider a pass as a “move”





Paper by Johnson, Aragon, McGeoch and Schevon on Bisectioning using SA

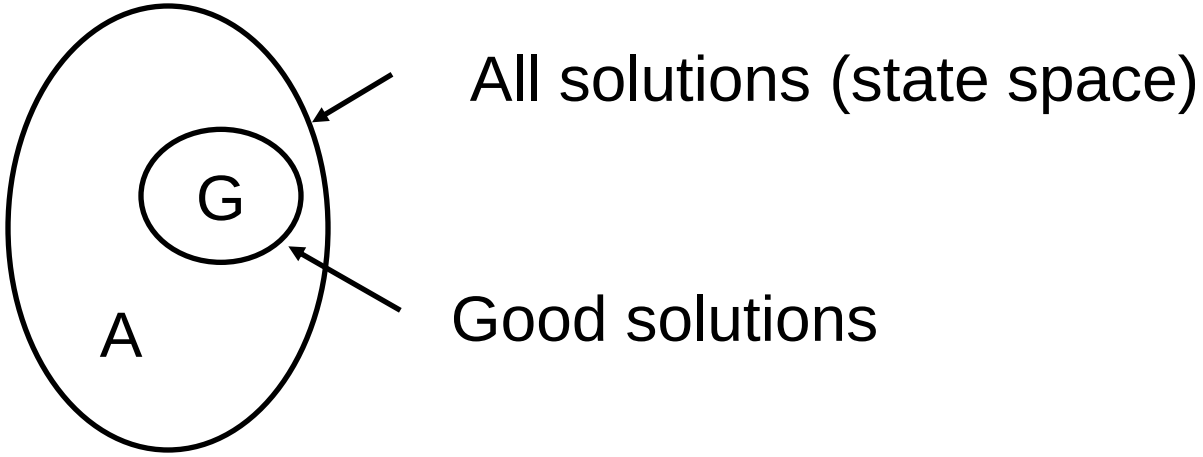
"Optimization by Simulated Annealing: An Experimental Evaluation Part I, Graph Partitioning", Operations Research, 37:865-892, 1989.

SA – The Evaluation

- An extensive empirical study of Simulated Annealing versus Iterative Improvement Approaches
- Conclusion: SA is a competitive approach, getting better solutions than KL for random graphs
- Remarks:
  - Netlists are not random graphs, but sparse graphs with local structure
  - SA is too slow. So KL/FM variants are still most popular
  - Multiple runs of KL/FM variants with random initial solutions may be preferable to SA

The Use of Randomness

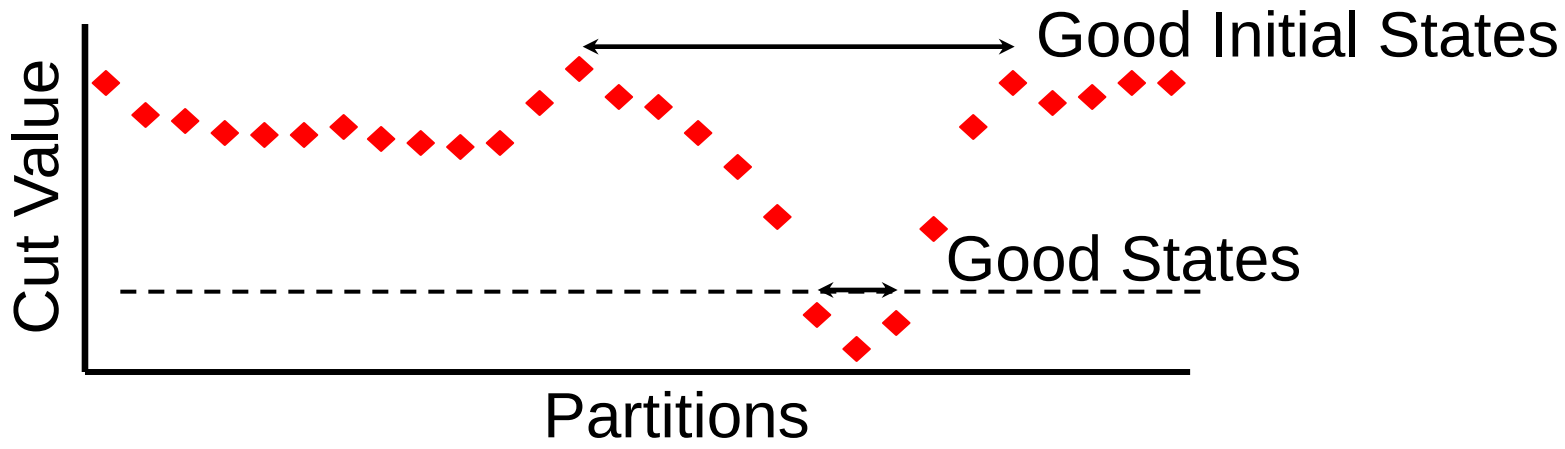
- For any partitioning problem:



- Suppose solutions are picked randomly
- If $|G|/|A| = r$, Pr(at least 1 good in $5/r$ trials) = $1-(1-r)^{5/r}$
- If $|G|/|A| = 0.001$, Pr(at least 1 good in 5000 trials) = $1-(1-0.001)^{5000} = 0.9933$

Adding Randomness to KL/FM

- In fact, # of good states are extremely few. Therefore, r is extremely small.
- Need extremely long time if just picking states randomly (without doing KL/FM).
- Running KL/FM variants several times with random initial solutions is a good idea.

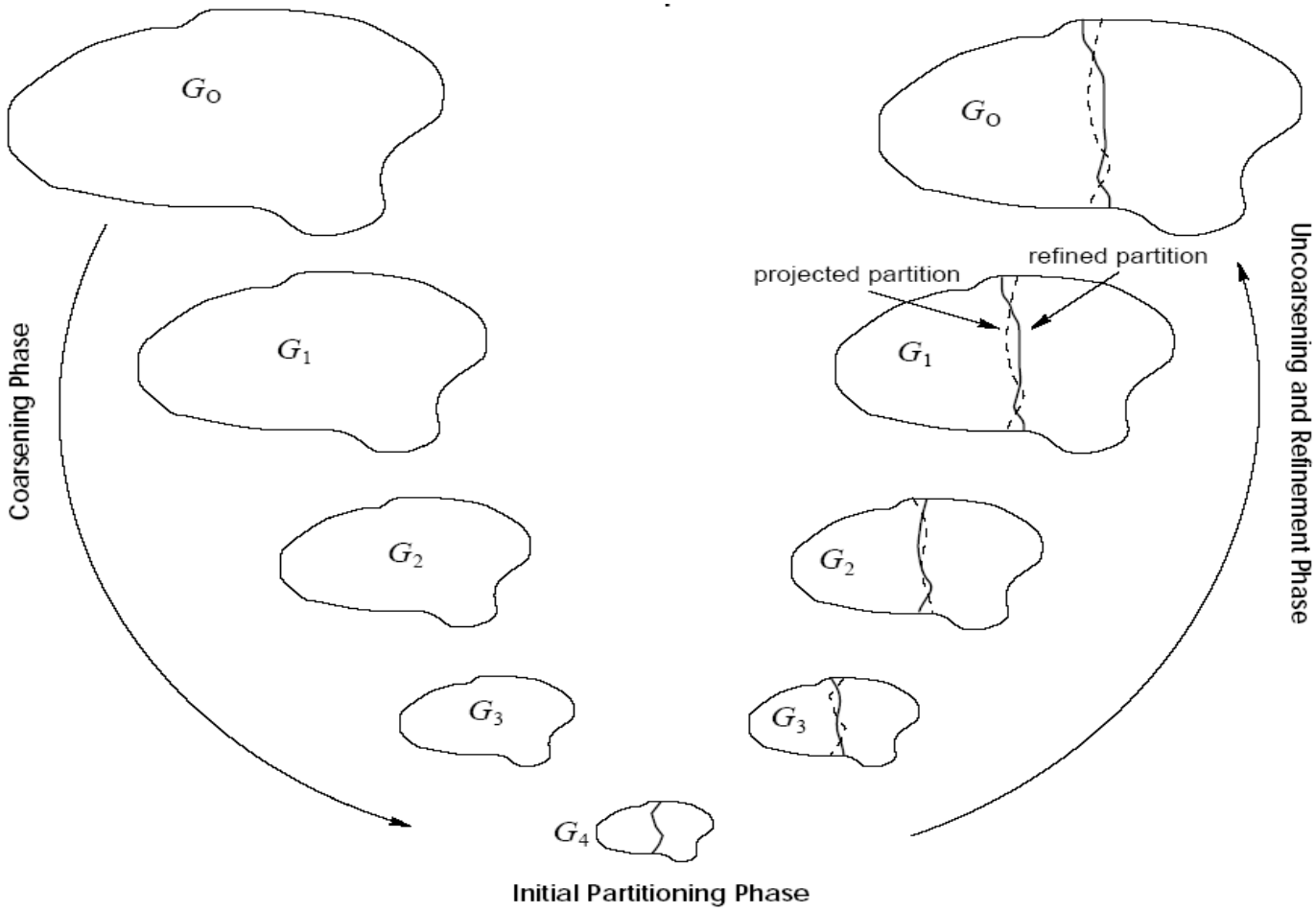


Some Other Approaches

- KL/FM-SA Hybrid: Use KL/FM variant to find a good initial solution for SA, then improve that solution by SA at low temperature
- Tabu Search
- Genetic Algorithm
- Spectral Methods (finding Eigenvectors)
- Network Flows
- Quadratic Programming
- …

Partitioning:
Multi-Level Technique

Multi-Level Partitioning

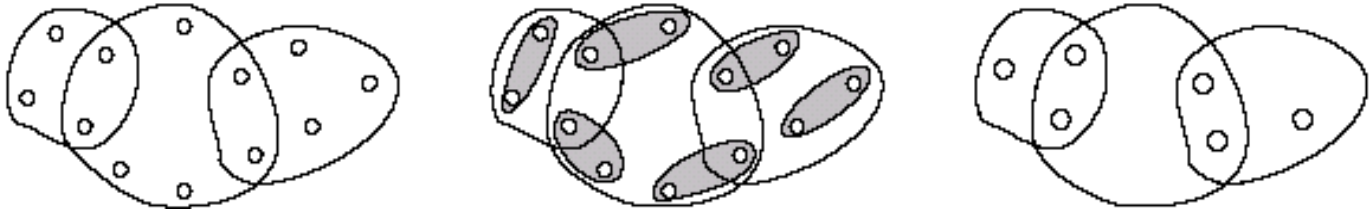


Multilevel Hypergraph Partitioning: Applications in VLSI Domain

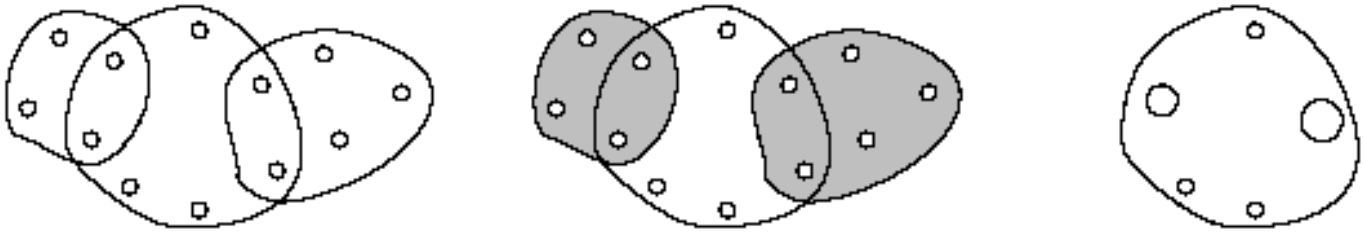
G. Karypis, R. Aggarwal, V. Kumar and S. Shekhar,
DAC 1997.

Coarsening Phase

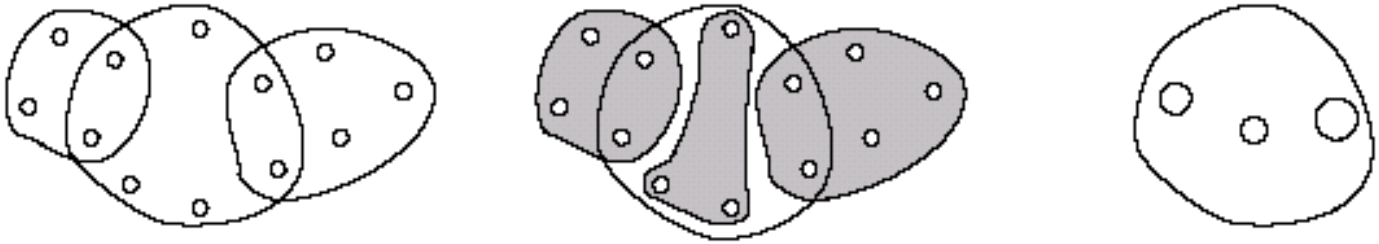
- Edge Coarsening



- Hyper-edge Coarsening (HEC)



- Modified Hyperedge Coarsening (MHEC)

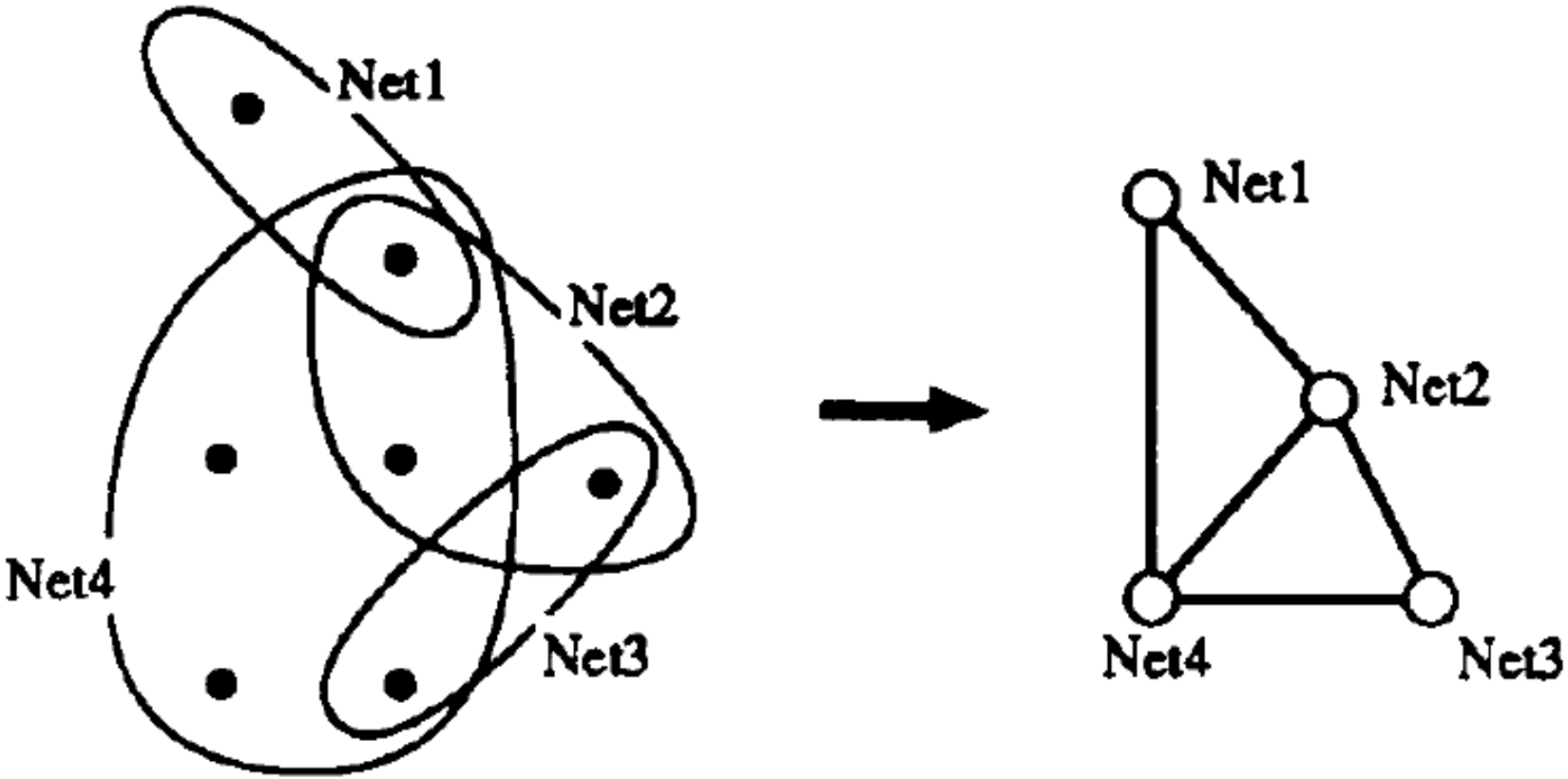


Uncoarsening and Refinement Phase

- FM:
  - Based on FM with two simplifications:
    - Limit number of passes to 2
    - Early-Exit FM (FM-EE), stop each pass if k vertex moves do not improve the cut

- HER (Hyperedge Refinement)
  - Move a group of vertices between partitions so that an entire hyperedge is removed from the cut

Net partitioning vs. Module partitioning



hMETIS Algorithm

- Software implementation available for free download from Web
- hMETIS-EE_{20}
  - 20 random initial partitions
  - with 10 runs using HEC for coarsening
  - with 10 runs using MHEC for coarsening
  - FM-EE for refinement
- hMETIS-FM_{20}
  - 20 random initial partitions
  - with 10 runs using HEC for coarsening
  - with 10 runs using MHEC for coarsening
  - FM for refinement

Experimental Results

- Compared with five previous algorithms
- hMETIS-EE_{20} is:
  - 4.1% to 21.4% better
  - On average 0.5% better than the best of the 5 algorithms
  - Roughly 1 to 15 times faster
- hMETIS-FM_{20} is:
  - On average 1.1% better than hMETIS-EE_{20}
  - Improve the best-known bisections for 9 out of 23 test circuits
  - Twice as slow as hMETIS-EE_{20}


© 2002 National Geographic Society. All rights reserved.
NATIONALGEOGRAPHIC.COM